Membres : Christophe TARATIBU, Mathieu Jaunet

Rapport Projet Compilation

Le Projet de l'UE Compilation consiste à créer un compilateur en réutilisant le projet d'analyse syntaxique. Dans ce cas-ci, la compilation prend charge le langage TPC qui est en fait un sous ensemble du langage C et doit renvoyer un fichier assembleur qui peut contenir des warnings mais pas d'erreur syntaxique, sémantique et lexicale. Ce rapport ci-joint exposera la création du compilateur et les difficultés rencontrées dans ce projet.

I. Arbre Abstrait, améliorations .

La grammaire représentative du langage tpc a éte légèrement changée pour le cas des variables locales statiques et déclarations de variables Pour le non-terminal DeclVars , deux règles ont été ajoutées :

1. Un terminal STATIC a été créé pour prendre en compte les variables statiques. Pour les La première règle ajoutée est DeclVars STATIC TYPE Declarateurs qui renvoie une erreur syntaxique pour déclaration de variable globale statique. La seconde règle qui est vide, permet éviter la duplication de nœuds lors de la conception de l'arbre abstrait. Ce non terminal est restreint aux variables globales.

- Dans le corps d'une fonction quelconque, un nouveau non-terminal DeclVarsLocale a été créée. Ses règles sont les mêmes de le non-terminal DeclVars. Elle a été créée pour séparer les variables globales et locales.
- Par la suite des actions ont été éditées pour construire l'arbre abstrait suivant les fonctions du programme tree.c (tree.h).
- Le fichier d'analyse lexical (lex.l) contient l'implémentation du lexique (lexèmes) supportés par le langage tpc avec les mots clés propres du langage c, les noms de variables et résultats (caractère, nombres) de ces dernières supportées et enfin les unités de langage propre au langage C ({}, +, -,&&...).
- Des nœuds inutiles ont été supprimés afin de simplifier l'arbre.

II. Implémentation du compilateur.

Pour crée le compilateur, le projet à été divisé en 3 parties : la création de la table des symboles qui se fait en un premier parcours de l'arbre abstrait en créant la

cette table. Cette même table contient un tableau de fonctions qui liste les fonctions présentes dans le code. Ce même tableau contient des éléments (tableau de paramètres et tableau local de paramètres) qui composent les fonctions. Un autre tableau contient les variables globales qui sont définis au tout début de la fonction. En se qui concerne la vérification sémantique du code tpc, elle est faite en parcourant une deuxième fois l'arbre l'abstrait. Cette partie affiche les warnings et erreurs rencontrés dans le code. Après la vérification que le programme cible ne contient aucune erreur, La génération du fichier asm est fait en reparcourant (encore une fois) l'arbre abstrait en écrivant dans le fichier les instructions nasm. Les fonctions utilitaires getint, putint, getchar et putchar sont intégré à la table des symboles de fonctions et écrites pour chaque code nasm crée.

III. Exécution

L'exécution du code final se passe en deux phases :

- Faire make (qui compile tous les fichiers nécessaires du projet)
- Faire ./bin/tpcc [OPTION] < [CHEMIN DU FICHIER] suivant les options choisies (-t pour afficher l'arbre abstrait, -h pour l'abstrait, -s, pour l'arbre des symboles et -c pour la génération du fichier asm)
- L'aide peut être consulté pour plus d'informations
- Pour tester les sorties des codes, faire python3 test.py

Le projet contient plusieurs dossiers et fichiers :

- Bin : dossier binaire
- Obj : dossier fichier objets
- Src : Dossier sources
- Tests : contenant les dossiers, good, syn-err, sem-err et warn et aussi les fichier asm relatif au dossier (sa présence sera expliqué dans la section problèmes rencontrés).
- Test.py qui teste les codes des dossiers
- Makefile : qui le fichier tpcc

IV. Problèmes rencontrés

Dans ce projet, des difficultés ont été rencontrés. Dans le banc de tests de sémantique disponible sur Elearning, la barre des 80% de tests réussi a été atteint mais tout à été fait pour atteindre la note maximale mais rien n'a changé après les

tests de projets. Pour le test de l'assembleur la compilation a été impossible. Dans le dossier de tests good des warnings de manque de return peuvent apparaître ce cas est la mal traité. Mais le plus important, les nombres négatives n'ont pas été prise en compte dans ce projet avec la difficulté des du changement de registres 32/64 bits.

V. Conclusion

En conclusion la majorité des consignes a été respecté même si les nombres négatifs n'ont pas été pris en compte, ceci impacte la présence du nombre de tests présent qui les réduit en conséquence. Néanmoins ce projet nous a beaucoup appris sur la conception d'un compilateur avec les différentes caractéristiques à prendre en compte